Laboratory Activity No. 9

Continuing the Flutter Web Project with a Report Form and UI State Responses

Reusable Screen and Widget Structure | Loading, Success, Empty, and Error States

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

IT 110 - Web Systems and Technologies

Instructor: JEAN VILLASIN, MIT

Student: Andy Mark L. Crasco (GitHub: andymarkcrasco-blip)

Project: activity8 - Event Venue and Micro-Vendor Finder With Machine Learning-Based Sentiment Analytics in Caraga Region

Date: October 2026

1. What we did

Activity 9 is a continuation, not a restart. We kept the Flutter Web project and the GitHub repository we built in Activity 8 and added to it. The Home, Login and Dashboard pages are still there and still work the way they did before, and the folder structure we set up in Activity 8 (screens, widgets, routes, services, models, constants) is exactly the same. What is new is lib/screens/report_form_screen.dart, a report form for our system where a client or a vendor can report a problem about a venue listing, a booking, a payment, or ask for a feature.

The form has five inputs - report title, category, location, description, and an optional contact number - and it reuses the AppTextField and AppButton widgets we made in Activity 8 instead of writing new UI code. The category and location values are specific to our project: the locations are the places in the Caraga region (Butuan City, Surigao del Norte, Surigao del Sur, Agusan del Norte, Agusan del Sur, Dinagat Islands) and the categories are the kinds of problems a venue or vendor can run into.

The part we spent the most time on was the four asynchronous responses the activity asks for. A report submission is not instant, so the screen had to show something while the request was running, something when it worked, something when there was nothing to show, and something when it failed. We wrote these as an enum in the screen itself, ReportUiState, with idle, loading, success, empty and error, and a separate AppStateView widget draws them so they all look like they belong to the same app.

One thing we want to be clear about: the responses are simulated. We are not connected to a server yet, so the wait is a Future.delayed in lib/services/report_service.dart. We did not want that to be hidden, so there is a switch on the form labelled "Simulate API failure (demo only)" that we can turn on during the demonstration to show the error and retry behaviour on purpose. The message on the success screen also says that the response is simulated.

Files we added and changed

- lib/screens/report_form_screen.dart - new. The form, the validation, and the four UI states.
- lib/widgets/app_state_view.dart - new. Draws the loading, success, empty and error responses.
- lib/services/report_service.dart - new. The simulated submit. It can also be told to fail.
- lib/models/report_model.dart - new. The report data object.
- lib/widgets/app_button.dart - we added an isLoading flag so the button can show a spinner and switch itself off while the request is running.

- lib/widgets/app_text_field.dart - we added maxLines, helperText and prefixIcon so the same field can be used for a single line and for the description box.
- lib/routes/app_routes.dart - we registered the /report route in the go_router setup we already had.
- lib/screens/home_screen.dart - we added a "Go to Report Form" button. The existing buttons are untouched.
- lib/widgets/app_shell.dart - we added Report to the navigation bar and the drawer.
- lib/constants/app_constants.dart - we moved the category and location lists here instead of keeping them inside the screen.

2. The four states and how we triggered them

Loading happens the moment the form is submitted with valid input. The submit button turns grey and shows a spinner, and the response area below the form says that the request is pending. The button is also disabled, which is what stops somebody from submitting the same report twice.

Success is what the screen shows once the request finishes without failing. We get a green confirmation with the reference number of the report we just created, and there is a button that takes us back to a clean form.

Empty is what we see when there is nothing to show. If we open the report screen before any report has been submitted, the response area says that there are no report records and the list underneath reads Submitted Reports (0). This is different from a validation error - validation is about one field being wrong, empty is about there being no data at all.

Error is triggered by turning on the simulate failure switch and submitting. The screen turns red, tells us the submission did not go through, and gives us a Retry button that runs the same submit again. We wanted the retry to actually work rather than just sit there, so the button calls the same function that the first submit used.

3. Problems we ran into while building it

The first issue was with the dropdown fields. At first we were using the label text to find them in our own tests, but the label changes once a value is chosen, so the tests could not find the field anymore. We changed the tests to target the dropdown by its position in the form instead, and after that the form tests passed.

The second issue was on the Git side. When we went to push Activity 9 we found the remote repository already had an earlier push from Activity 8 that only contained the starter files, and our two earlier commits had been made with a placeholder email address. We rewrote the author on those commits so that they show up under the right GitHub account, and then we pushed our branch as main and kept master in sync with it. We also had to point Git Credential Manager at our own account, because there was more than one GitHub account saved on the computer and it kept asking us which one to use.

The third issue was small but annoying: on a 1366x768 screen the response area was below the fold, so the loading and error messages were cut off in our screenshots. We scrolled the page before capturing so that the whole response box and the action button are visible in the evidence.

4. How we ran and tested it

```
flutter pub get
flutter run -d chrome
flutter build web
```

The app runs in Chrome on the Home, Login, Dashboard and Report screens. The demo login is student / 1234 and it is local only, there is no backend behind it yet. We also wrote automated tests for the important parts of the screen - that

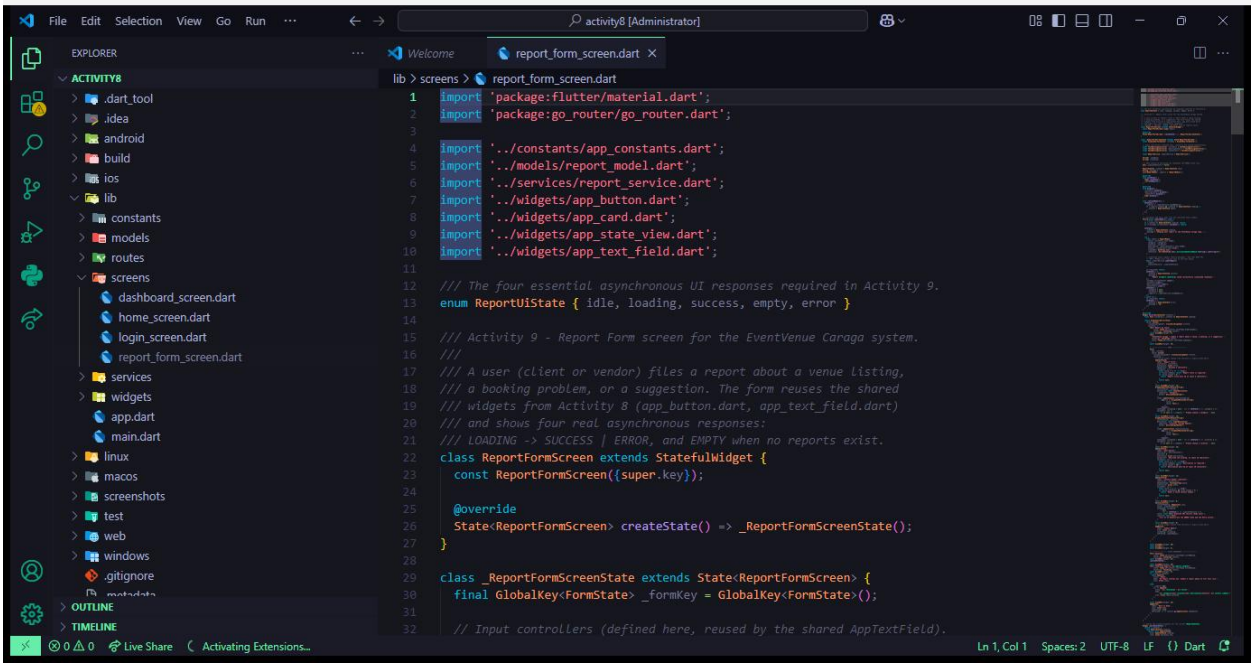
the home page still links to the report form, that the empty state appears when there are no records, that invalid input is rejected field by field, that a valid submission goes from loading to success, and that a failed submission shows the error with a working retry. All six tests pass and flutter analyze reports no issues.

5. Screenshot evidence

The six required screenshots are below. We added one extra at the start showing the home page, because that is where the navigation into the report form happens.

Screenshot 1 - the updated project structure

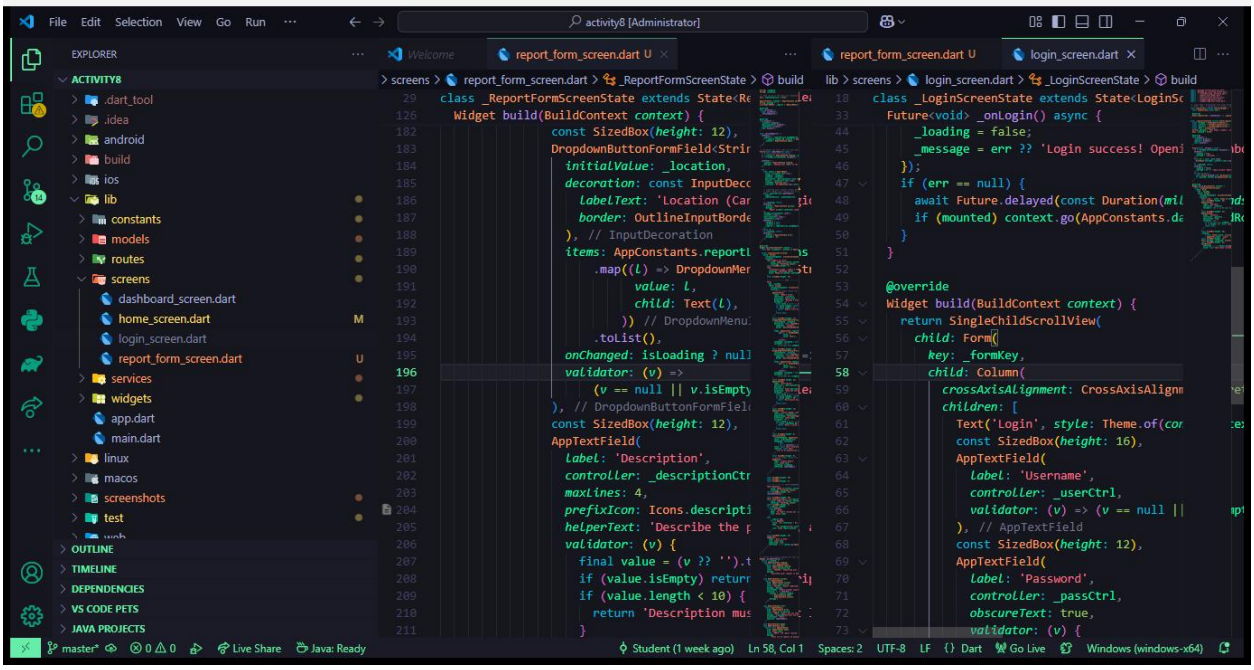
This is VS Code with the project open. On the left you can see the folders we set up in Activity 8, and under screens the new report_form_screen.dart sitting next to home, login and dashboard. The file is open on the right with the imports at the top, including the widgets we are reusing.



Screenshot 1 - VS Code Explorer showing the Activity 8 folders and the new report_form_screen.dart

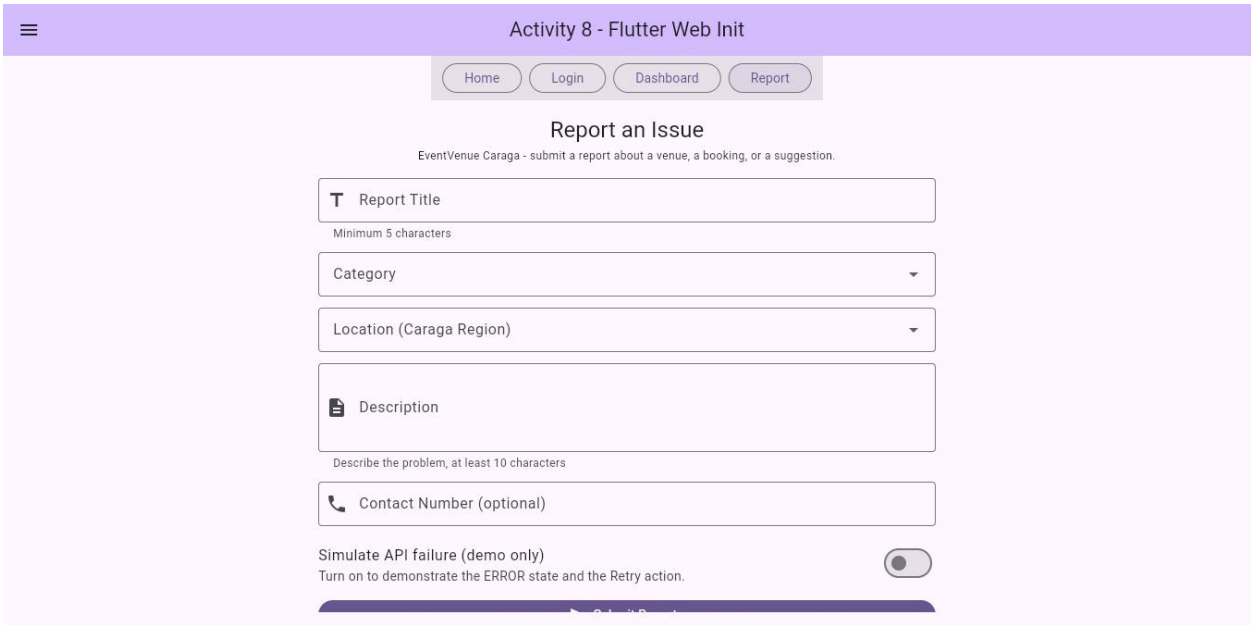
Screenshot 2 - the shared widget used in two different screens

The activity asks us to show a widget from lib/widgets/ being used in the report form and in another screen. On the left is report_form_screen.dart at the Description field, and on the right is the Activity 8 login_screen.dart at the Username field. Both are the same AppTextField class from lib/widgets/app_text_field.dart. AppButton is shared the same way, and it is used in four screens: home, login, dashboard and the report form.



Screenshot 2a - AppTextField used in report_form_screen.dart (left) and login_screen.dart (right)

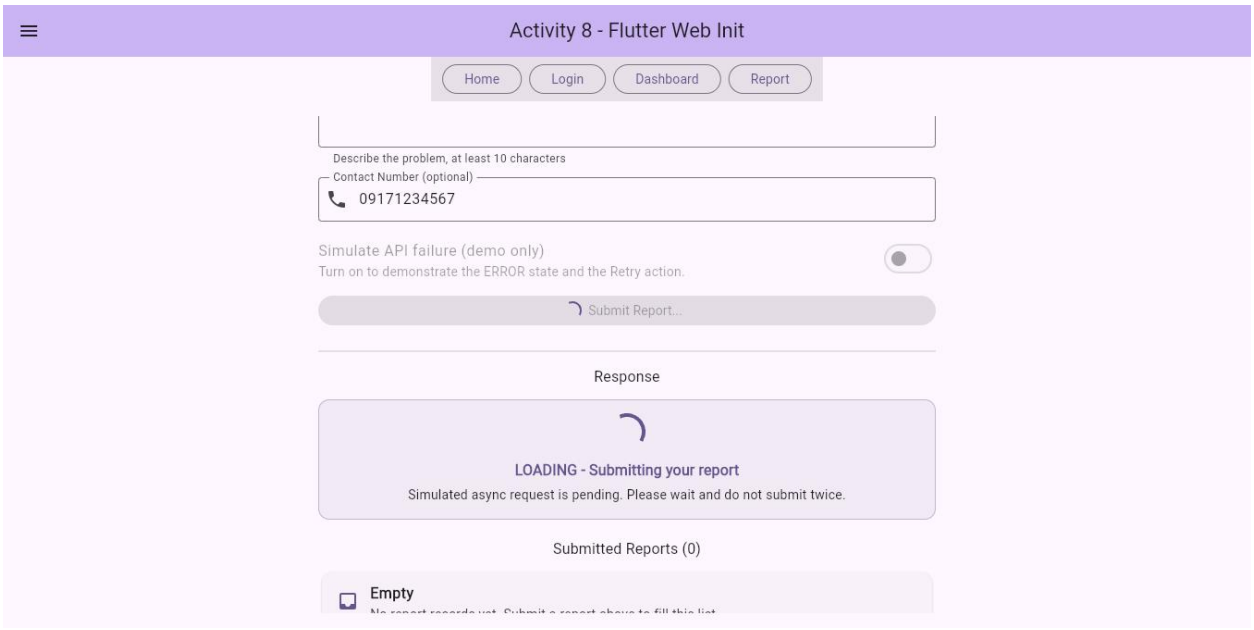
The same screen running in Chrome, so you can see the form as it actually looks with the fields and the shared buttons.



Screenshot 2b - report_form_screen.dart running in Google Chrome

Screenshot 3 - loading

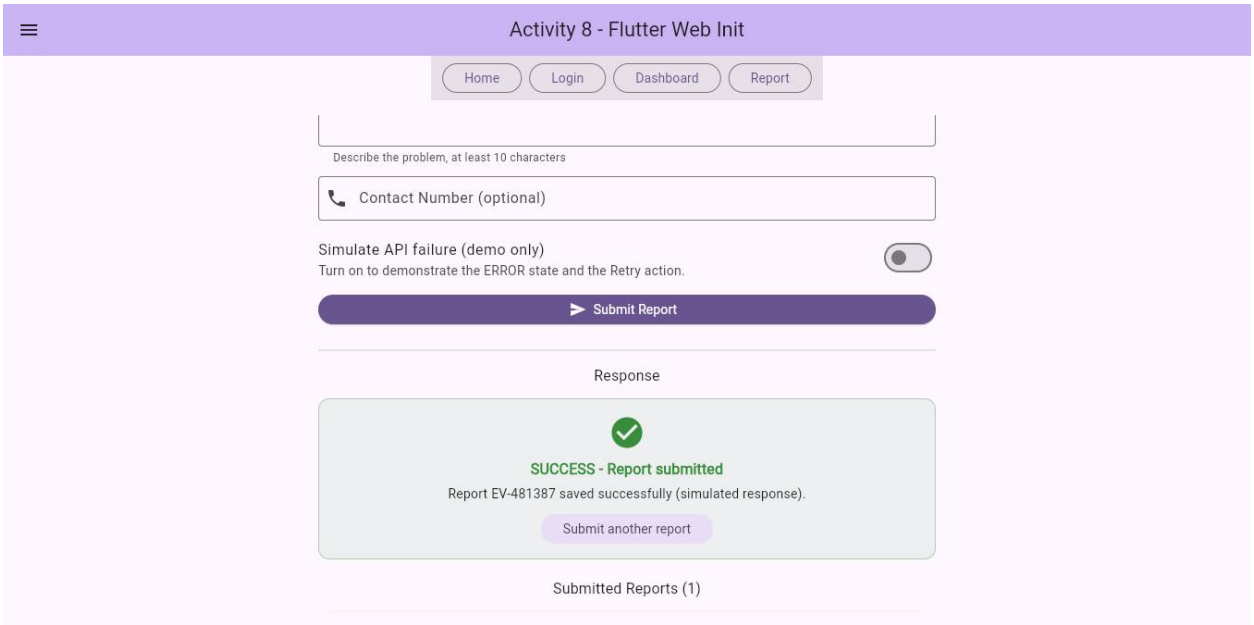
We filled in the form with a valid report - a beach resort venue in Butuan City that closed early on the event date - and pressed Submit Report. This was captured while the request was still pending. The button is disabled and spinning, and the response area shows the loading message.



Screenshot 3 - Loading state, with the submit button disabled so the report cannot be sent twice

Screenshot 4 - success

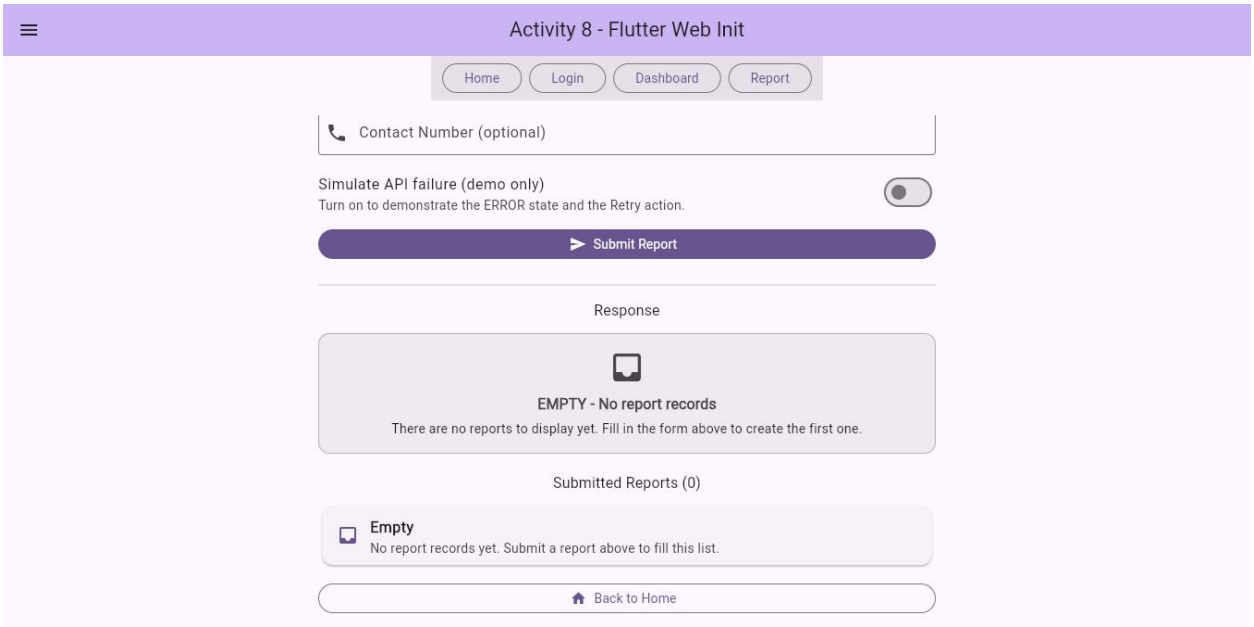
A few seconds later the simulated request finished without failing. The response turned green and shows the reference number of the report that was created, with a button to submit another one. The form fields were cleared, which is why they are empty again in this screenshot.



Screenshot 4 - Success state with the saved report reference number

Screenshot 5 - empty

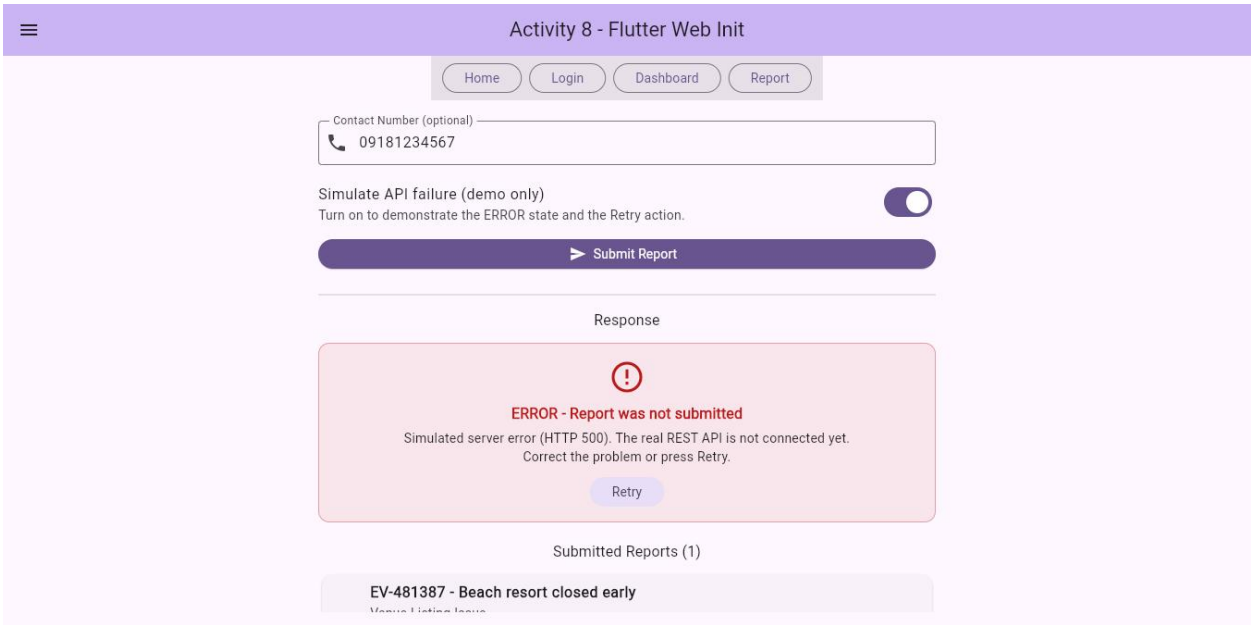
This was taken before anything was submitted, so the response area says there are no report records and the list below shows Submitted Reports (0). This is the no-data response, not a validation error.



Screenshot 5 - Empty state, no report records to display yet

Screenshot 6 - error

We turned on the simulate failure switch and submitted again. The screen went red and explained that the report was not submitted, and the Retry button is right there underneath. The report we created successfully earlier is still in the list below, so we can see the successful and failed submissions side by side.



Screenshot 6 - Error state with the visible Retry action

6. Verification checklist

Test	What we observed
Home to Report Form navigation	Works. The button on the home page and the Report button in the navigation bar both open the form (Screenshots 1 and 2b).
Invalid form is rejected	Works. Empty title gives "Report title is required", empty category "Please choose a category", empty location "Please choose a location", empty description "Description is required". Covered by an automated test.
Valid submit goes loading then success	Works, Screenshots 3 and 4. Covered by an automated test.
No records gives the empty response	Works, Screenshot 5. Covered by an automated test.
Failure gives error and retry	Works, Screenshot 6. The retry button runs the submit again. Covered by an automated test.
Shared widget used in two places	AppTextField appears in report_form_screen.dart and login_screen.dart, shown side by side in Screenshot 2a. AppButton is used in four screens.

7. Repository and checkpoints

We used the same repository and the same branch as Activity 8, so Activity 9 is simply the next commit in the history rather than a new project.

```
git status
git add .
git commit -m "Add report form and Flutter UI states"
git push
```

- Repository: https://github.com/andymarkcrasco-blip/activity8
- c7497c3 - Initial Flutter Web project - home/login/navigation shell (Activity 8)
- 1ef6ac3 - Add report form and Flutter UI states (Activity 9)
- 00af0f8 - Post-laboratory reflection answers to evidence report (Activity 9)

8. Limits of what we have done

We want to be honest about where the project stands. There is no server behind this yet, so nothing we submit is really stored anywhere - the reports live in memory for as long as the app is running. Every one of the four states is simulated, including the error, and the switch on the form makes that explicit rather than hiding it. The REST API (an instructor-approved backend such as FastAPI) and the PostgreSQL database from IT 108 are still only planned.

When we connect them, the change is mostly in one file. The Future.delayed inside report_service.dart becomes an http.post to the endpoint, the JSON that comes back is mapped into ReportModel, and the screen asks the API for the list of reports so that the empty state comes from a real query. The enum and the four builders in report_form_screen.dart do not have to change at all, which is the reason we kept the state handling separate from the form in the first place.

9. Post-laboratory reflection

1. How does report_form_screen.dart extend the Activity 8 project without replacing the Home and Login screens?

It sits alongside them instead of taking their place. We created it as a new file in lib/screens/ and registered a new /report route in the go_router configuration that Activity 8 already had. The Home and Login screens were only given one extra button each so we could reach the form; we did not rewrite their logic or their layout. Because the structure from Activity 8 is still in place, the new screen was able to reuse the shared widgets straight away instead of duplicating them.

2. Which shared widget did we reuse, and what benefit does it give us?

We reused AppTextField, in the report form and in the login screen, and AppButton, which is used in four screens. The benefit is that we only build it once. The label style, the border, the validation hook and the disabled-while-loading behaviour are all defined in one place, so when we change the look of the field every screen changes with it. It also keeps the code smaller and means two of us can work on different screens at the same time without stepping on each other's files.

3. How does the form prevent invalid input and duplicate submissions?

For invalid input we use a GlobalKey<FormState> with a validator on each field. The title needs at least five characters, the description at least ten, the category and location have to be chosen from the dropdowns, and a contact number, if given, needs at least seven digits. For duplicate submissions we check the state at the start of submitReport and return immediately if it is still loading, and on top of that the shared AppButton is given isLoading so it is drawn disabled with a spinner while the request is running.

4. How did we reproduce the loading, success, empty and error responses, and which of them are simulated?

All four are simulated at the moment. Loading was captured while the delayed request was still pending. Success was captured after it finished without failing. Empty was captured by opening the form before anything had been submitted. Error was captured by turning on the simulate failure switch, which makes the service throw the way a real server error would. The point of the switch is that when the class is demonstrated we can show the error on purpose instead of waiting for something to break.

5. What has to change when the real REST API and PostgreSQL database are connected?

Only lib/services/report_service.dart. The delayed wait is replaced with an http.post call to the backend, the response is mapped into ReportModel, and the screen loads the report list from the API so that the empty state is based on a real query. Everything in report_form_screen.dart stays as it is, including the enum and the four response builders, so the four states carry over to the real backend without any rework.